

D1 · DELIVERABLE

White-Box Test Report

Control-flow graph, cyclomatic complexity, independent paths, the silent-parse-failure test and coverage evidence.

Structural testing of the specified screening routine and of the parser defect the case study identifies. Every number is produced by the test run and the corpus evaluation used to build this document.

Contents

1	Scope and unit under test	2	Control-flow graph
3	Cyclomatic complexity	4	Independent paths and test cases
5	Boundary values, equivalence classes and metamorphic tests	6	The silent parse failure test
7	Coverage evidence	8	Defects found during testing (bug reports)
9	Test inventory		

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project Management	D1 · v1.0
Roll 150096724029	B.Tech CSE · Batch 2024-28 · Semester V	1 October 2026 · ITM Skills University

1 Scope and unit under test

This report applies white-box (structural) testing to the screening routine that the case study specifies, and to the defect the case study names as the real one: résumé parsing that **fails silently** on 29% of documents. The routine is implemented as one pure function, so its control flow can be read straight from the code.

Specification (case study, verbatim)

```
if mandatory skills missing, reject;
else score experience band;
  if score ≥ threshold and location matches, shortlist;
  else if score ≥ threshold and remote allowed, shortlist;
  else waitlist;
if parse confidence < 0.6, route to manual review.
```

Implementation under test

`api/src/ranking/routine.ts` → `screen(parsed, criteria)`. The function is pure: it has no I/O and no clock. Each statement carries the CFG node id it implements, and every call returns a **trace** of the nodes it visited. That trace is stored with the decision in the append-only log.

Test framework: `bun test` (Bun 1.3, Jest-compatible), with coverage from `bun test --coverage`. Results come from the run used to build this document.

```
export function screen(p: ParsedResume, c: Criteria): Decision {
  const missingSkills = c.mandatorySkills.filter((s) => !p.skills.includes(s)); // N1
  if (missingSkills.length > 0) { outcome = "reject" } // N2
  else {
    ({ score, band } = bandScore(p.yearsExperience, c.bands)); // N3
    const meets = score >= c.threshold;
    const locMatch = !!p.location && p.location.toLowerCase() === c.location.toLowerCase();
    if (meets && locMatch) outcome = "shortlist"; // N4 → N5
    else if (meets && c.remoteAllowed) outcome = "shortlist"; // N6 → N7
    else outcome = "waitlist"; // N8
  }
  if (p.confidence < CONFIDENCE_GATE) { // N9 (0.6)
    tentativeOutcome = outcome; outcome = "manual_review"; // N10
  }
  return { outcome, tentativeOutcome, score, band, reasonCode, reason, trace }; // N11
}
```

TESTS RUN

82

165 assertions

FAILURES

0

0 skipped · 2.5 s

ROUTINE.TS COVERAGE

100%

lines · 100% functions

CFG PATHS COVERED

8 / 8

all complete paths, 5 =
basis set

2 Control-flow graph

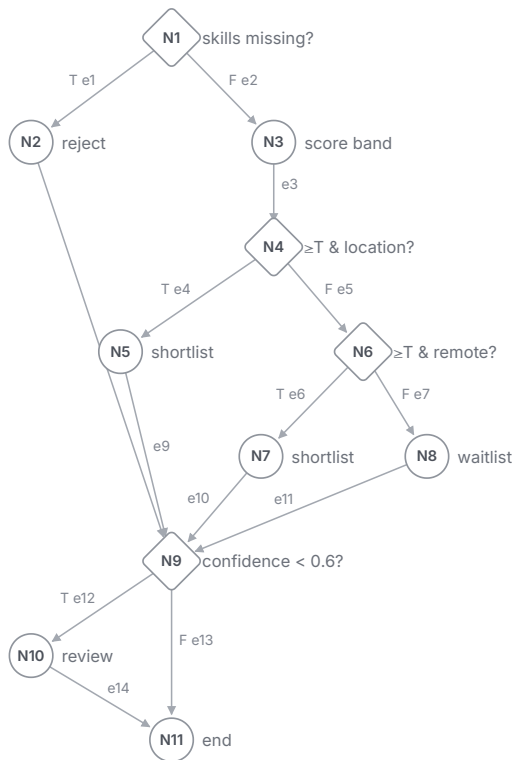


Figure 1. CFG of `screen()`. Diamonds are predicate (decision) nodes. Edges e1–e14 are labelled T/F where they leave a decision.

Table 1. Nodes and the statements they represent

NODE	STATEMENT	TYPE
N1	if mandatory skills missing	decision
N2	reject	process
N3	else score experience band	process
N4	if score $\geq$ threshold and location matches	decision
N5	shortlist	process
N6	else if score $\geq$ threshold and remote allowed	decision
N7	shortlist	process
N8	else waitlist	process
N9	if parse confidence < 0.6	decision
N10	route to manual review	process
N11	end (decision logged)	process

Table 2. Edges

EDGE	FROM $\rightarrow$ TO	CONDITION
e1	N1 $\rightarrow$ N2	true
e2	N1 $\rightarrow$ N3	false
e3	N3 $\rightarrow$ N4	—
e4	N4 $\rightarrow$ N5	true
e5	N4 $\rightarrow$ N6	false
e6	N6 $\rightarrow$ N7	true
e7	N6 $\rightarrow$ N8	false
e8	N2 $\rightarrow$ N9	—
e9	N5 $\rightarrow$ N9	—
e10	N7 $\rightarrow$ N9	—
e11	N8 $\rightarrow$ N9	—
e12	N9 $\rightarrow$ N10	true
e13	N9 $\rightarrow$ N11	false
e14	N10 $\rightarrow$ N11	—

Why N9 comes after every branch. The case study lists the confidence rule last, so it applies to every outcome, including a rejection. This turns the silent failure into a *withheld* decision. An unreadable résumé looks like "no skills" at N1, and the old system rejected it. Here N9 overrides that rejection, sends the application to a person, and keeps the automated outcome as "tentative" for the reviewer.

3 Cyclomatic complexity

M McCabe's cyclomatic complexity $V(G)$ is the number of linearly independent paths through the graph, so it is the minimum number of tests that exercises every edge. It is computed three independent ways:

METHOD	FORMULA	VALUES	V(G)
Edges and nodes	$V(G) = E - N + 2P$	$E = 14, N = 11, P = 1$ connected component	5
Predicate nodes	$V(G) = \pi + 1$	$\pi = 4$ (N1, N4, N6, N9)	5
Regions	$V(G) = \text{enclosed regions} + 1$	4 enclosed regions of the planar graph + 1 outer region	5

Condition-level view (compound predicates)

N4 and N6 are compound conditions ($\text{score} \geq T \wedge \text{location}, \text{score} \geq T \wedge \text{remote}$). Splitting them into atomic conditions gives six predicates: N1, N4a $\text{score} \geq T$, N4b location matches, N6a $\text{score} \geq T$, N6b remote allowed and N9. That makes $V(G) = 7$ at condition level. One combination is **infeasible**: N4a = false with N6a = true is impossible, because both test the same expression. The implementation therefore evaluates $\text{meets} = \text{score} \geq \text{threshold}$ once and reuses it. The condition-coverage tests in §5 exercise every feasible value of every atomic condition.

Result: $V(G) = 5$. At least 5 test cases are needed for edge (branch) coverage. The routine has 8 complete entry-to-exit paths: 4 outcomes before N9, times 2 for N9. The test suite covers all 8.

4 Independent paths and test cases

Basis set P1–P5. Each path adds at least one edge not used by the previous ones, so together they cover all 14 edges. P6–P8 complete the path set.

PATH	NODE SEQUENCE	MEANING	TEST (FILE:LINE)	RESULT
P1	N1 → N2 → N9 → N11	Mandatory skill missing, confident parse → reject	routine.test.ts:10	Pass
P2	N1 → N3 → N4 → N5 → N9 → N11	Score $\geq$ threshold, in job city → shortlist	routine.test.ts:18	Pass
P3	N1 → N3 → N4 → N6 → N7 → N9 → N11	Score $\geq$ threshold, elsewhere, remote allowed → shortlist	routine.test.ts:25	Pass
P4	N1 → N3 → N4 → N6 → N8 → N9 → N11	Below threshold → waitlist	routine.test.ts:32	Pass
P5	N1 → N2 → N9 → N10 → N11	Parse failed (no skills read, confidence 0) → manual review, not reject	routine.test.ts:39	Pass
P6 extra	N1 → N3 → N4 → N5 → N9 → N10 → N11	Would shortlist, but confidence < 0.6 → review	routine.test.ts:56	Pass
P7 extra	N1 → N3 → N4 → N6 → N7 → N9 → N10 → N11	Would shortlist (remote), confidence < 0.6 → review	routine.test.ts:62	Pass
P8 extra	N1 → N3 → N4 → N6 → N8 → N9 → N10 → N11	Would waitlist, confidence < 0.6 → review	routine.test.ts:68	Pass

Test data for the basis paths

Criteria fixture: mandatory skills `react, typescript`; bands 0–2 → 20, 2–4 → 55, 4–7 → 85, 7+ → 90; threshold 80; job city Bengaluru; remote allowed, unless a row says otherwise.

TC	PATH	SKILLS	YEARS	CITY	CONFIDENCE	CRITERIA CHANGE	EXPECTED OUTCOME / REASON CODE
TC-01	P1	react	5	Bengaluru	0.90	—	reject · MISSING_MANDATORY_SKILLS (typescript)

TC	PATH	SKILLS	YEARS	CITY	CONFIDENCE	CRITERIA CHANGE	EXPECTED OUTCOME / REASON CODE
TC-02	P2	react, typescript, css	5	Bengaluru	0.90	—	shortlist · SHORTLIST_LOCATION
TC-03	P3	react, typescript, css	5	Pune	0.90	—	shortlist · SHORTLIST_REMOTE
TC-04	P4	react, typescript, css	3	Bengaluru	0.90	—	waitlist · WAITLIST_BELOW_THRESHOLD (score 55)
TC-05	P5	— (none read)	unknown	unknown	0.00	—	manual_review · LOW_PARSE_CONFIDENCE, tentative reject
TC-06	P6	react, typescript, css	5	Bengaluru	0.45	—	manual_review, tentative shortlist
TC-07	P7	react, typescript, css	5	Pune	0.40	—	manual_review, tentative shortlist
TC-08	P8	react, typescript, css	3	Bengaluru	0.40	—	manual_review, tentative waitlist
TC-09	N6 F (location)	react, typescript, css	5	Pune	0.90	remote not allowed	waitlist · WAITLIST_LOCATION

5 Boundary values, equivalence classes and metamorphic tests

INPUT	EQUIVALENCE CLASSES	BOUNDARY TESTS (VALUE → EXPECTED)	RESULT
score vs threshold	below · at · above	score 85, threshold 85 → shortlist (≥ is inclusive); threshold 86 → waitlist	Pass
parse confidence	< 0.6 · ≥ 0.6	0.60 → decided automatically; 0.59 → manual review	Pass
years vs band edge	[2,4) · [4,7)	4.00 → score 85; 3.99 → score 55	Pass
years unknown	null	null → score 0 (never an error)	Pass
mandatory list	empty · non-empty	no mandatory skills → nothing can be missing	Pass
location text	case variants	"bengaluru" = "Bengaluru"	Pass
location unknown	null	null never matches the job city	Pass

Condition coverage (atomic conditions of N4 and N6)

MEETS (SCORE ≥ T)	LOCATION MATCHES	REMOTE ALLOWED	OUTCOME	COVERED BY
true	true	—	shortlist (N5)	TC-02
true	false	true	shortlist (N7)	TC-03
true	false	false	waitlist, location	TC-09
false	— (short-circuit)	— (short-circuit)	waitlist, below threshold	TC-04
false at N4, true at N6			infeasible: same expression	—

Metamorphic tests: attributes outside the criteria cannot change a decision

The case study's internal review found the old ranking tracked **university tier**. The relation tested here is: changing only the university (IIT Bombay ↔ Anna University ↔ none), or only the name and email, must leave the outcome and the score unchanged. Pass Pass

6 The silent parse failure test

The case study names the current system's real defect: *"résumé parsing works on 71% of documents and fails silently on the rest."* A silent failure is worse than a crash. The parser returns **empty skills** with no error, and the routine then rejects a possibly qualified candidate with a reason that looks legitimate: "missing mandatory skills". The tests below catch exactly that, on real PDFs produced by the seed templates.

TEST	INPUT	ASSERTION	RESULT
Legacy rejects silently	scanned résumé (glyph outlines, no text layer) of a candidate who has every skill	legacy parse → skills = [] and no error; legacy outcome = reject (reproduces the defect)	Pass
v2 detects it	same PDF	v2 textChars < 200 → confidence = 0, issue "no usable text layer"; routine → manual_review , tentative reject	Pass
Unreadable section	experience written in seasons ("Summer 2023 – Spring 2025")	years = null but the section has content → confidence capped at 0.50 → manual review	Pass
OCR recovery	same scanned PDF, OCR enabled (PP-OCrv6)	text recovered, skills/years/city correct, confidence ≤ OCR word confidence → decided automatically	Pass

System-level evidence on the 2,000-résumé corpus

LEGACY CORRECT 71.0% case-study figure: 71%	LEGACY SILENT FAILURES 580 every failure is silent	HIRESENSE CORRECT 99.0% 14 of 20 failures routed to review	HIRESENSE SILENT 6 0.30% of résumés
--	---	---	--

A parse is *correct* only when all three decision fields match ground truth: skills ($F1 \geq 0.8$), years (± 1) and city. The corpus is synthetic and built so that 29% of résumés use layouts that break the legacy parser.

7 Coverage evidence

Line and function coverage from `bun test --coverage`, for the modules that make decisions. Bun does not report branch coverage, so edge (branch) coverage of the routine comes from the path tests in §4 (all 14 edges). It is confirmed independently from production traces in the table below.

MODULE	% FUNCTIONS	% LINES	UNCOVERED LINES
src/audit/fairness.ts	100.0	100.0	–
src/audit/service.ts	100.0	100.0	–
src/db.ts	0.0	60.0	119–122
src/legacy/legacy.ts	100.0	97.4	–
src/parsing/dates.ts	100.0	100.0	–
src/parsing/llm.ts	85.7	93.3	47–53
src/parsing/ocr.ts	100.0	95.0	21–22
src/parsing/parser.ts	100.0	100.0	–

MODULE	% FUNCTIONS	% LINES	UNCOVERED LINES
src/parsing/pdf.ts	100.0	100.0	–
src/parsing/taxonomy.ts	100.0	100.0	–
src/ranking/order.ts	100.0	95.2	–
src/ranking/routine.ts	100.0	100.0	–
src/scoring/experience.ts	100.0	100.0	–
src/screening.ts	100.0	100.0	–

The remaining uncovered lines are I/O and environment wrappers: the database bootstrap, the "is the local model running" probe, and the OCR-unavailable branch. The HTTP layer (`src/index.ts`) is exercised by API smoke tests and is not part of the unit run.

Table 3. CFG paths actually taken by the 2,170 decisions in the append-only log

PATH	NODE SEQUENCE	DECISIONS
P1	N1 → N2 → N9 → N11	673
P4	N1 → N3 → N4 → N6 → N8 → N9 → N11	604
P2	N1 → N3 → N4 → N5 → N9 → N11	550
P3	N1 → N3 → N4 → N6 → N7 → N9 → N11	313
P8	N1 → N3 → N4 → N6 → N8 → N9 → N10 → N11	17
P5	N1 → N2 → N9 → N10 → N11	13

Edges exercised in production: **14 / 14**. P6 and P7 do not occur naturally in the corpus: a confident parse rarely sits next to a shortlist-level score. That is why they are covered by targeted unit tests.

8 Defects found during testing (bug reports)

Every defect below was found by a test, by the evaluation against ground truth, or by observing the running system. Each one was fixed and is now guarded by a regression test.

ID	SEVERITY	FOUND BY	DEFECT AND ROOT CAUSE	FIX / REGRESSION TEST
BUG-01	High	Corpus evaluation	Letter-spaced headings were extracted as "S K I L L S" and not recognised. Both parsers failed 721 standard résumés, and education years were counted as experience.	Collapse single-letter runs before heading match · <i>letter-spaced headings</i>
BUG-02	Med	Corpus evaluation	City detection returned the first city in list order rather than in text order (95 wrong cities when a university name contained a city).	Earliest occurrence wins · parser tests
BUG-03	High	Corpus evaluation	Two-column layouts whose sidebar held < 15% of the text were not detected, so 24 résumés merged their columns.	Balance floor 8% + side-by-side baseline check · <i>two-column</i>
BUG-04	High	Path P5 review	Season dates ("Summer 2023") gave years = null yet confidence 0.70. A new silent failure .	Anomaly cap 0.50 when a section has content but yields nothing · <i>unreadable season dates</i>
BUG-05	Low	AI assist review	"Node.js" also matched JavaScript through the "JS" alias.	Lookbehind treats "." as part of a word · <i>Node.js does not also match</i>
BUG-06	High	OCR benchmark	OCR emits U+2212 "-" and drops spaces ("Sep2021-Jul2023"), so 37 of 160 scanned résumés lost a job.	Dash set and month/year spacing widened · date-range tests
BUG-07	Med	Test writing	AI assist kept a hallucinated city whenever the résumé mentioned any other city.	City must be the one named in the text · <i>drops hallucinated</i>

ID	SEVERITY	FOUND BY	DEFECT AND ROOT CAUSE	FIX / REGRESSION TEST
BUG-08	Med	Manual testing	AI assist turned a headline plus an education year into a fake job (employer "N/A").	Employer must appear in the résumé · <i>a headline or education line is not a job</i>
BUG-09	Med	Manual testing	The local model took longer than Bun's 10 s idle timeout and the response was cut off.	idleTimeout 120 s
BUG-10	Med	User report	PDF viewer re-rendered endlessly: the scrollbar toggled the width between 444 and 429 px.	scrollbar-gutter: stable + ignore scrollbar-sized resizes
BUG-11	Med	Report writing	Fairness reported a vacuous ratio of 1.0 when only one group had $n \geq 30$.	Ratio = none + "insufficient sample" · <i>only one comparable group</i>

9 Test inventory

All 82 automated tests from the run used for this report (JUnit output of `bun test`).

FILE	SUITE	TEST	RESULT
routine.test.ts	basis paths (V(G) = 5)	P1 N1→N2→N9 → N11: mandatory skill missing, confident parse → reject	Pass
routine.test.ts	basis paths (V(G) = 5)	P2 N1→N3→N4→N5→N9 → N11: score $\geq$ threshold and location matches → shortlist	Pass
routine.test.ts	basis paths (V(G) = 5)	P3 N1→N3→N4→N6→N7→N9 → N11: score $\geq$ threshold, elsewhere, remote allowed → shortlist	Pass
routine.test.ts	basis paths (V(G) = 5)	P4 N1→N3→N4→N6→N8→N9 → N11: below threshold → waitlist	Pass
routine.test.ts	basis paths (V(G) = 5)	P5 ...→N9→N10 → N11: parse confidence < 0.6 → manual review (the silent-failure path)	Pass
routine.test.ts	remaining branch outcomes	score $\geq$ threshold, elsewhere, remote NOT allowed → waitlist for location	Pass
routine.test.ts	remaining branch outcomes	low confidence overrides a shortlist too (automated outcome is only tentative)	Pass
routine.test.ts	remaining branch outcomes	remote shortlist withheld by low confidence: N1→N3→N4→N6→N7→N9→N10	Pass
routine.test.ts	remaining branch outcomes	waitlist withheld by low confidence: N1→N3→N4→N6→N8→N9→N10	Pass
routine.test.ts	remaining branch outcomes	unknown location never matches the job city	Pass
routine.test.ts	boundary values	score exactly at threshold is shortlisted ($\geq$, not $>$)	Pass
routine.test.ts	boundary values	score one below threshold is waitlisted	Pass
routine.test.ts	boundary values	confidence exactly 0.6 is NOT routed to review (< 0.6)	Pass
routine.test.ts	boundary values	confidence 0.59 is routed to review	Pass
routine.test.ts	boundary values	band edges: 4.0 yrs falls in [4,7), 3.99 falls in [2,4)	Pass
routine.test.ts	boundary values	unparsed experience scores 0	Pass
routine.test.ts	boundary values	no mandatory skills configured → nothing can be missing	Pass
routine.test.ts	boundary values	location match is case-insensitive	Pass
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	university = IIT Bombay	Pass
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	university = Anna University	Pass

FILE	SUITE	TEST	RESULT
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	university = null	Pass
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	name and email do not affect the decision	Pass
routine.test.ts		every decision carries a human-readable reason and a trace ending at the confidence gate	Pass
audit.test.ts	4/5ths impact-ratio rule	ratio exactly 0.80 passes	Pass
audit.test.ts	4/5ths impact-ratio rule	ratio 0.78 breaches	Pass
audit.test.ts	4/5ths impact-ratio rule	groups under 30 are reported but cannot breach	Pass
audit.test.ts	4/5ths impact-ratio rule	'Prefer not to say' is never compared	Pass
audit.test.ts	4/5ths impact-ratio rule	applications still in manual review are excluded from rates	Pass
audit.test.ts	4/5ths impact-ratio rule	a small group cannot set the benchmark rate	Pass
audit.test.ts	4/5ths impact-ratio rule	only one comparable group → no ratio (not a vacuous 1.0)	Pass
audit.test.ts	4/5ths impact-ratio rule	threshold constant is the case-study value	Pass
audit.test.ts	breach severity	no breach → pass	Pass
audit.test.ts	breach severity	breach that could be noise (n = 40, ratio 0.75) → warning	Pass
audit.test.ts	breach severity	significant breach (n = 300, ratio 0.70) → critical	Pass
audit.test.ts	rank order is explicit and explainable	same outcome and score → earlier application ranks higher	Pass
audit.test.ts	rank order is explicit and explainable	higher score wins within the same outcome	Pass
audit.test.ts	rank order is explicit and explainable	outcome dominates score	Pass
audit.test.ts	decision log is append-only	UPDATE is refused	Pass
audit.test.ts	decision log is append-only	DELETE is refused	Pass
parser.test.ts	standard résumé	v2 extracts every decision field with full confidence	Pass
parser.test.ts	standard résumé	legacy parser also handles the standard layout	Pass
parser.test.ts	silent parse failure — the current system's real defect	scanned/outlined PDF: legacy silently REJECTS a qualified candidate	Pass
parser.test.ts	silent parse failure — the current system's real defect	scanned/outlined PDF: v2 detects it (confidence 0) and routes to manual review	Pass
parser.test.ts	silent parse failure — the current system's real defect	unreadable season dates cap confidence below the gate instead of scoring 0 years silently	Pass
parser.test.ts	layouts the legacy parser gets wrong	two-column: v2 reads columns separately	Pass
parser.test.ts	layouts the legacy parser gets wrong	creative headings: v2 knows 'Toolbox' / 'Where I've Worked'; legacy does not	Pass
parser.test.ts	date ranges	Jan 2019 – Jan 2022 → 3 yrs	Pass
parser.test.ts	date ranges	January 2020 - Present → 6 yrs	Pass
parser.test.ts	date ranges	03/2019 – 03/2021 → 2 yrs	Pass
parser.test.ts	date ranges	2019/03 → now → 6.8 yrs	Pass
parser.test.ts	date ranges	'18 – '22 → 4 yrs	Pass

FILE	SUITE	TEST	RESULT
parser.test.ts	date ranges	Since Jan 2024 → 2 yrs	Pass
parser.test.ts	date ranges	03.2017 - 03.2020 → 3 yrs	Pass
parser.test.ts	date ranges	2016 – 2019 → 3 yrs	Pass
parser.test.ts	date ranges	Mar 2019 – Mar 2022 → 3 yrs	Pass
parser.test.ts	date ranges	Jul2022-Jan2024 → 1.5 yrs	Pass
parser.test.ts	date ranges	overlapping jobs are not double counted	Pass
parser.test.ts	date ranges	future and inverted ranges are ignored	Pass
parser.test.ts		letter-spaced headings ('S K I L L S') are still recognised	Pass
parser.test.ts	OCR fallback (scanned PDFs)	recovers the text of a scanned résumé and passes the gate	Pass
parser.test.ts	half-credit fallbacks (fields found outside their section)	skills without a skills heading count at half weight	Pass
parser.test.ts	half-credit fallbacks (fields found outside their section)	dates without an experience heading count at half weight	Pass
parser.test.ts		parseResume parses a rendered PDF end to end	Pass
parser.test.ts		legacy score rewards university tier (the bias the internal review found)	Pass
audit-service.test.ts	runAudit	fair outcomes → pass, automated rejection stays on	Pass
audit-service.test.ts	runAudit	significant breach → critical, auto-reject paused, later rejections routed to review	Pass
audit-service.test.ts	runAudit	small-sample breach → warning, not paused	Pass
audit-service.test.ts	runAudit	a passing re-audit resumes automated rejection	Pass
audit-service.test.ts	runAudit	no comparable groups → reported as insufficient sample	Pass
llm.test.ts	toMonths	2023-04	Pass
llm.test.ts	toMonths	Apr 2023	Pass
llm.test.ts	toMonths	April 2023	Pass
llm.test.ts	toMonths	Spring 2023	Pass
llm.test.ts	toMonths	Winter 2020	Pass
llm.test.ts	toMonths	2019	Pass
llm.test.ts	toMonths	present	Pass
llm.test.ts	toMonths	today	Pass
llm.test.ts	toMonths	unreadable wording → null	Pass
llm.test.ts	grounding	keeps grounded values and converts season dates	Pass
llm.test.ts	grounding	a headline or education line is not a job (employer must appear in the résumé)	Pass
llm.test.ts	grounding	drops hallucinated skills, cities and positions	Pass
llm.test.ts		'Node.js' does not also match JavaScript's 'JS' alias	Pass